```
  class UsersController < ApplicationController
    def index
      @users = User.all
    end
  end
end
```

and

```
class Admin::UsersController < ApplicationController
  def index
    @users = User.all
  end
end
```

To resolve `User` Ruby checks `Admin` in the former case, but it does not in the latter because it does not belong to the nesting (see Nesting and Resolution Algorithms).

Unfortunately Rails autoloading does not know the nesting in the spot where the constant was missing and so it is not able to act as Ruby would. In particular, `Admin::User` will get autoloaded in either case.

Albeit qualified constants with `class` and `module` keywords may technically work with autoloading in some cases, it is preferable to use relative constants instead:

```
module Admin
  class UsersController < ApplicationController
    def index
      @users = User.all
    end
  end
end
```

10.2 Autoloading and STI

Single Table Inheritance (STI) is a feature of Active Record that enables storing a hierarchy of models in one single table. The API of such models is aware of the hierarchy and encapsulates some common needs. For example, given these classes:

```
# app/models/polygon.rb
class Polygon < ActiveRecord::Base
end

# app/models/triangle.rb
class Triangle < Polygon
end

# app/models/rectangle.rb
class Rectangle < Polygon
end
```

`Triangle.create` creates a row that represents a triangle, and `Rectangle.create` creates a row that represents a rectangle. If `id` is the ID of an existing record, `Polygon.find(id)` returns an object of the correct type.

Methods that operate on collections are also aware of the hierarchy. For example, `Polygon.all`returns all the records of the table, because all rectangles and triangles are polygons. Active Record takes care of returning instances of their corresponding class in the result set.

Types are autoloaded as needed. For example, if `Polygon.first` is a rectangle and `Rectangle` has not yet been loaded, Active Record autoloads it and the record is correctly instantiated.

All good, but if instead of performing queries based on the root class we need to work on some subclass, things get interesting.